

SOLUTIONS PART 2

REVERSE SOLUTION

```
public String reverse(String str)
{
    if (str.isEmpty())
        return str;
    //Calling Function Recursively
    return reverse(str.substring(1)) +
str.charAt(0);
}
```

IS PALINDROME SOLUTION

```
public boolean isPalindrome(String s)
{    // if length is 0 or 1 then String
is palindrome
    if(s.length() == 0 || s.length() ==
1)

        return true;
    if(s.charAt(0) ==
s.charAt(s.length()-1))
        return isPalindrome(s.substring(1,
s.length()-1));
    return false;
}
```

SOME RECURSIVE SOLUTION

```
public boolean someRecursive(int[] arr,
OddFunction odd) {
    if (arr.length == 0 ) {
        return false;
    } else if (odd.run(arr[0]) == false) {
        return
someRecursive(Arrays.copyOfRange(arr, 1,
arr.length), odd);
    } else {
        return true;
    }
}
```

```
    }  
}
```

FIRST UPPERCASE SOLUTION

```
public static char first(String str) {  
    if(str.isEmpty()) {  
        return ' ';  
    }  
    if (Character.isUpperCase(str.charAt(0))) {  
        return str.charAt(0);  
    }else {  
        return first(str.substring(1,  
str.length()));  
    }  
}
```

CAPITALIZE WORD SOLUTION

```
public static String capitalizeWord(String  
str) {  
  
    if(str.isEmpty()) {  
        return str;  
    }  
    char chr = str.charAt(str.length()-1);  
    if(str.length()==1) {  
        return  
Character.toString(Character.toUpperCase(chr))  
;  
    }  
    if((str.substring(str.length()-2,  
str.length()-1).equals(" "))) {  
        chr = Character.toUpperCase(chr);  
    }  
    return  
capitalizeWord(str.substring(0,str.length()-1)  
) + Character.toString(chr);  
}
```

